

Source Artifact Model

Preserving Immutable Healthcare Evidence Before FEN Interpretation

Phoros Architecture Note

August 21, 2026

The idea in one sentence

A `SourceArtifact` does not contain document bytes. It is an immutable Rust domain record that reaches those bytes through a typed `StoredObjectRef`; processing runs interpret the preserved bytes, and FEN facts record the resulting semantic assertions.

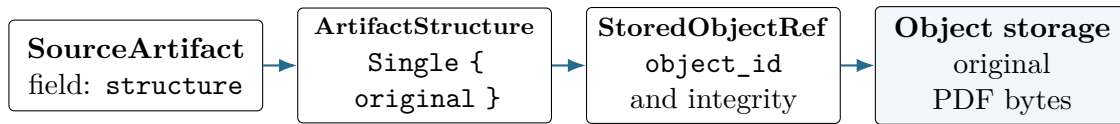
Contents

1	Begin With One PDF	2
2	Why Separate the Record From the Bytes?	3
3	From Preserved Bytes to FEN Facts	4
4	Identity Is Not a Content Hash	4
5	The Stored-Byte Boundary	5
6	The Immutable Core	5
7	Extending the PDF Model: Single and Compound	6
8	Receipt Provenance and Authority	7
9	Processing Is Separate and Repeatable	9
10	Citing Exact Evidence	9
11	Worked Example: Epic EHI Export	10
12	Lifecycle Without Mutation	11
13	Domain Invariants	11
14	Security and Privacy Boundaries	12
15	Relationship to FEN	12
16	Repository Boundaries	13
17	Decisions to Make Later	13
18	Recommended Implementation Sequence	14
A	Consolidated Initial Rust Schema	14
B	Review Checklist	18

1 Begin With One PDF

Suppose Phoros receives a file named `medical-record.pdf`. Before considering parsers, FEN facts, or compound archives, ask the simplest question: *where do the PDF bytes live?*

The answer is: in protected object storage. Neither `SourceArtifact` nor `StoredObjectRef` contains the PDF bytes inside the Rust value.



Read the diagram from left to right:

1. `SourceArtifact` records the receipt event: account, time, provenance, authority, supplied metadata, and structure.
2. Its `structure` field is `ArtifactStructure::Single` because the received source is one PDF.
3. The variant's `original` field is a `StoredObjectRef`. This small Rust value identifies the stored object and carries its digest, length, media type, and protection reference.
4. A storage adapter uses that reference to retrieve and decrypt the actual PDF bytes from object storage.

The most important boundary

The `SourceArtifact` contains an `ArtifactStructure`; the structure contains a `StoredObjectRef`; object storage contains the PDF bytes. The word “contains” means something different at each step.

1.1 The nesting in Rust

The relevant types, stripped of unrelated fields, are:

```

pub struct StoredObjectRef {
    pub object_id: StoredObjectId,
    pub digest: ContentDigest,
    pub byte_length: u64,
    pub media_type: MediaType,
    pub protection: ProtectionRef,
}

pub enum ArtifactStructure {
    Single {
        original: StoredObjectRef,
    },
    // Compound is introduced later.
}

pub struct SourceArtifact {
    // receipt context omitted here
    pub structure: ArtifactStructure,
}
  
```

Conceptually, a received PDF therefore has this shape:

```
SourceArtifact {
  structure: ArtifactStructure::Single {
    original: StoredObjectRef {
      object_id: /* lookup key for protected storage */,
      digest: /* hash of the exact PDF bytes */,
      byte_length: /* size of the PDF */,
      media_type: /* application/pdf */,
      protection: /* how access is authorized */,
    },
  },
  // receipt context omitted here
}
```

The retrieval path is thus:

```
artifact.structure
  -> Single.original
  -> StoredObjectRef.object_id
  -> object-store lookup and authorized decryption
  -> original PDF bytes
```

1.2 Four similar terms with different meanings

Term	What it is
SourceArtifact	The immutable domain record for one received source event. It has provenance and a nested reference to the original bytes.
ArtifactStructure	The typed description of whether the source is one object or a compound container.
StoredObjectRef	A small Rust value that identifies protected bytes and records how to verify them. It is not the byte payload.
Stored object	The protected byte object held by the storage system; for this example, it is the exact PDF that arrived.

2 Why Separate the Record From the Bytes?

Healthcare information reaches a patient in forms designed by many different institutions: PDFs, portal downloads, email attachments, claims files, FHIR bundles, and large electronic health information (EHI) exports. Phoros needs to preserve these sources before it attempts to understand them.

The separation lets the domain model remain small and testable while a storage adapter handles streaming, encryption, authorization, and provider-specific object locations. It also prevents large or sensitive payloads from being copied through ordinary domain operations.

That ordering matters. A parser can be improved. A coding rule can change. An extraction model can be replaced. The bytes that actually arrived must remain stable so that every later interpretation can be reproduced, challenged, or corrected.

The source artifact model therefore has three responsibilities:

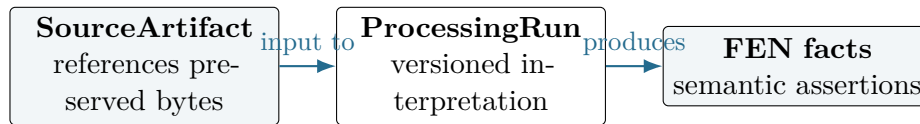
1. preserve exactly what was received;

2. record how, when, and under what authority it was received; and
3. provide precise citations for processing results and FEN facts.

It deliberately does *not* decide what the source means clinically or economically. Meaning belongs to processing and FEN.

3 From Preserved Bytes to FEN Facts

After the PDF boundary is clear, the larger model has three stages: preserved evidence, repeatable interpretation, and semantic state.



A **SourceCitation** connects a produced fact back through its processing run to the precise artifact, and optionally to a page, archive member, row, or field.

Object	Question it answers
SourceArtifact	What evidence did Phoros receive, for which account, and with what provenance?
StoredObjectRef	Which protected byte object holds the source, and how can those bytes be verified?
ProcessingRun	Which parser or model interpreted the evidence, using which version and configuration?
SourceCitation	What exact part of the evidence supports this output?
FEN fact	What semantic assertion did Phoros accept into the patient record?

4 Identity Is Not a Content Hash

A content hash is excellent for integrity, but it is the wrong identity for a source artifact.

Suppose the same PDF arrives once from a provider portal and later as an email attachment. The bytes may be identical, but Phoros observed two distinct deliveries with different timestamps, channels, and authorization histories. Those deliveries should produce two **SourceArtifactId** values even if storage deduplicates their bytes.

```

pub struct SourceArtifactId(pub String);
pub struct StoredObjectId(pub String);

pub struct ContentDigest {
    pub algorithm: DigestAlgorithm,
    pub value: Vec<u8>,
}

pub enum DigestAlgorithm {
    Sha256,
}
  
```

Identity rule

SourceArtifactId identifies an evidentiary occurrence. **ContentDigest** identifies byte content. Storage may deduplicate content without collapsing receipt history.

5 The Stored-Byte Boundary

The domain model should not expose filesystem paths, cloud URLs, credentials, raw encryption keys, or an in-memory byte vector. It retains only a provider-neutral reference and the information needed to verify the retrieved bytes.

```
pub struct StoredObjectRef {
    pub object_id: StoredObjectId,
    pub digest: ContentDigest,
    pub byte_length: u64,
    pub media_type: MediaType,
    pub protection: ProtectionRef,
}

pub struct MediaType {
    pub value: String,
}

pub struct ProtectionRef(pub String);
```

The `object_id` is resolved by a storage adapter; it is not necessarily a path or URL. The digest is computed over the exact original plaintext bytes before encryption. Encryption metadata and key resolution belong to the storage adapter.

The digest itself should still be treated as sensitive operational metadata because a known-file attack can reveal that an account holds a particular document.

6 The Immutable Core

The central aggregate is intentionally small. It records stable receipt context and a nested route to the evidence; it does not embed the evidence payload or conclusions produced later.

```
pub struct SourceArtifact {
    pub id: SourceArtifactId,
    pub account_id: AccountId,
    pub received_at: Timestamp,
    pub receipt: ReceiptProvenance,
    pub authorization: AcquisitionAuthority,
    pub structure: ArtifactStructure,
    pub supplied_metadata: SuppliedMetadata,
}
```

There is deliberately no `pdf: Vec<u8>` or `bytes: Vec<u8>` field. For the PDF example, the route to the bytes begins at `structure`, continues through `ArtifactStructure::Single.original`, and ends at the referenced stored object.

6.1 Why the account is required but the subject is not

The receiving endpoint belongs to a Phoros account, so account ownership is known at receipt time. The people or other semantic subjects mentioned in a document may not yet be known. Determining them is an interpretation step.

Requiring `SubjectId` during receipt would encourage the ingestion layer to guess. Subject associations should instead be recorded later with their evidence and confidence.

6.2 What immutability means

Once finalized, the aggregate is never edited in place. In particular, the following remain stable:

- the original object reference and digest;
- the trusted receipt timestamp and receiving endpoint;
- the metadata supplied with the delivery; and
- the acquisition authority recorded at receipt.

Later knowledge is appended as a separate observation. For example, a sender may be claimed at receipt and verified later. The verification does not rewrite the original claim.

7 Extending the PDF Model: Single and Compound

The PDF example establishes the single case. A ZIP export changes only the `ArtifactStructure` variant; the surrounding `SourceArtifact` still records one receipt event.

```
pub enum ArtifactStructure {
    Single {
        original: StoredObjectRef,
    },
    Compound {
        original: StoredObjectRef,
        manifest: ArtifactManifest,
    },
}
```

For a PDF, `Single.original` is the `StoredObjectRef` for the exact PDF bytes. For an Epic EHI ZIP, `Compound.original` is the `StoredObjectRef` for the exact ZIP bytes, while `manifest` describes a verified decomposition of the archive.

What stays authoritative

The original reference always resolves to exactly what arrived: PDF bytes in the single case and ZIP bytes in the compound case. A manifest and any extracted member objects improve inspection and processing; they do not replace the original archive as evidence.

```
pub struct ArtifactManifest {
    pub format: ContainerFormat,
    pub members: Vec<ArtifactMember>,
}
```

```
pub enum ContainerFormat {  
    Zip,  
    Tar,  
    GzipTar,  
    Other(String),  
}
```

```
pub struct ArtifactMember {  
    pub id: ArtifactMemberId,  
    pub relative_path: String,  
    pub uncompressed_length: u64,  
    pub digest: ContentDigest,  
    pub detected_media_type: Option<MediaType>,  
    pub extracted_object: Option<StoredObjectRef>,  
}
```

The member digest proves the extracted bytes. The optional extracted-object reference allows efficient processing without making the extracted copy a new source event. The archive remains authoritative.

7.1 Why a manifest is useful

For a large EHI export, the manifest enables Phoros to:

- verify that extraction produced the expected members;
- process one table without repeatedly expanding the archive;
- detect missing, duplicate, or changed members;
- cite a stable member identity instead of relying on a filename alone; and
- preserve directory relationships and documentation shipped with the export.

Archive inspection must be bounded. Member counts, expanded byte totals, path traversal, nested archives, and compression ratios require defensive limits before an artifact is finalized as compound.

8 Receipt Provenance and Authority

Provenance must distinguish what the sender claimed from what Phoros verified.

```
pub struct ReceiptProvenance {  
    pub channel: ReceiptChannel,  
    pub endpoint_id: CommunicationsEndpointId,  
    pub external_message_id: Option<String>,  
    pub claimed_sender: Option<String>,  
    pub transport_evidence: Vec<TransportEvidenceRef>,  
}
```

```
pub enum ReceiptChannel {  
    SecureUpload,  
    Smtplib,  
    DirectMessaging,  
    Fhir,  
    ProviderApi,  
    LocalImport,  
}
```

```
pub enum AcquisitionAuthority {  
    PatientProvided,  
    PatientDirectedDisclosure,  
    HipaaRightOfAccess,  
    AuthorizedAccountManager,  
    Other(AuthorityEvidenceRef),  
}
```

The exact authority taxonomy may evolve. The important initial rule is that acquisition authority is typed and evidence-bearing rather than an unstructured note.

Claim versus verification

An email header may claim a sender. A Direct signature or provider API credential may verify one. Store the claim in the immutable receipt record and append verification results separately; never silently promote a claim into a verified identity.

9 Processing Is Separate and Repeatable

A processing run records one attempt to interpret an artifact. It is not part of the immutable artifact because the same source can be processed repeatedly as software improves.

```
pub struct ProcessingRun {
  pub id: ProcessingRunId,
  pub artifact_id: SourceArtifactId,
  pub processor: ProcessorIdentity,
  pub configuration_ref: Option<String>,
  pub started_at: Timestamp,
  pub completed_at: Option<Timestamp>,
  pub outcome: ProcessingOutcome,
}

pub struct ProcessorIdentity {
  pub name: String,
  pub version: String,
}

pub enum ProcessingOutcome {
  Running,
  Succeeded,
  SucceededWithWarnings,
  Failed { error_code: String },
}
```

Processing should be reproducible from the artifact digest, processor version, and configuration. Nondeterministic systems, including AI models, should additionally record the model identity and any sampling or prompt-profile information needed to explain the result.

10 Citing Exact Evidence

A source citation joins a processing result to the evidence that supports it.

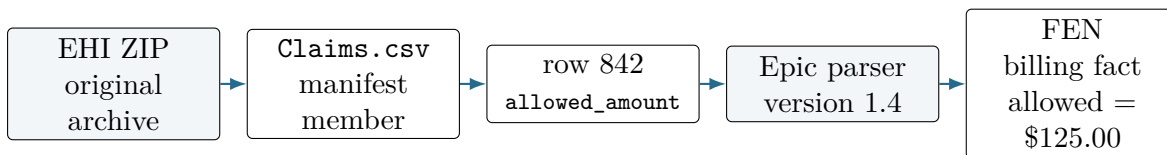
```
pub struct SourceCitation {
  pub artifact_id: SourceArtifactId,
  pub member_id: Option<ArtifactMemberId>,
  pub location: SourceLocation,
  pub processing_run_id: ProcessingRunId,
}
```

```
pub enum SourceLocation {
  WholeArtifact,
  PdfPage {
    page_number: u32,
  },
  DelimitedField {
    data_row: u64,
    column_name: String,
  },
  JsonPointer {
    pointer: String,
  },
  ByteRange {
    start: u64,
    length: u64,
  },
}
```

The citation is format-aware enough to be useful to a reviewer but does not embed parsed healthcare meaning. It says where evidence is located, not what that evidence means.

11 Worked Example: Epic EHI Export

Assume an account receives `patient-export.zip`. The archive contains `Claims.csv`, and row 842 has an `allowed_amount` value of 12500 minor currency units.



The resulting fact cites the artifact, the `Claims.csv` member, row 842, the named column, and processing run 1.4.

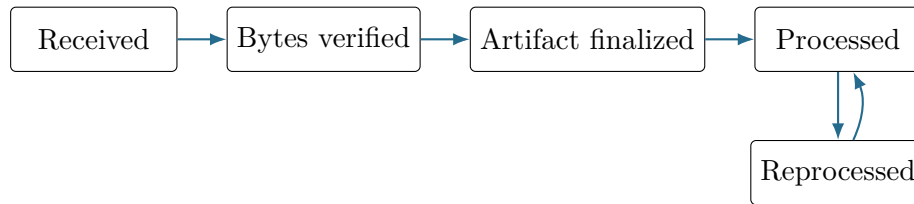
Later, parser version 1.5 discovers that the table's currency column must also be consulted. It creates a new processing run and may produce a corrected or superseding FEN fact. Nothing about the original ZIP, manifest, or row is rewritten.

The audit question

For any derived FEN fact, Phoros should be able to answer: “Which exact evidence did we read, and which versioned process turned it into this assertion?”

12 Lifecycle Without Mutation

Immutability applies to the evidence record, not to every operational decision around it.



Receipt may use a short-lived staging record while bytes are uploaded, hashed, scanned, and—for archives—safely inspected. The immutable **SourceArtifact** is committed atomically only after its object references and manifest are complete.

Quarantine, access suspension, retention schedules, legal holds, and deletion are separate lifecycle records. They do not mutate the evidentiary claim about what arrived.

12.1 Deletion and tombstones

“Immutable” should mean unchangeable while retained, not impossible to delete when law, policy, or patient direction requires deletion. A purge operation should remove the protected bytes and append a non-sensitive audit tombstone containing the artifact ID, deletion authority, time, and outcome. It must not imply that the bytes remain retrievable.

13 Domain Invariants

The first implementation should enforce the following rules at construction time:

1. Every artifact has an opaque identity and a owning account.
2. Every stored object has a tagged digest and byte length.
3. A finalized artifact references only successfully persisted objects.
4. A compound artifact retains the exact original container.
5. Every manifest member has a safe relative path, stable member identity, digest, and length.
6. Member identities are unique within a manifest.
7. Extraction limits are checked before finalization.
8. A processing run identifies one exact artifact and processor version.
9. A citation’s member belongs to the cited artifact.
10. Storage deduplication never merges source-artifact receipt records.

These invariants belong in constructors and repository boundaries rather than being left to application convention.

14 Security and Privacy Boundaries

Source artifacts will often contain direct identifiers and dense protected health information. The source domain should therefore assume that content and much of its metadata are sensitive.

- Encrypt original and extracted objects with account-scoped protection.
- Encrypt manifests when filenames or directory structure can reveal health information.
- Keep searchable operational metadata minimal.
- Require authorization before issuing a read capability or decryption operation.
- Audit access to originals independently from access to derived FEN facts.
- Treat malware scanning and archive inspection as untrusted-input boundaries.
- Never place storage credentials or public object URLs in domain values.

The domain can define a `SourceArtifactRepository` and an object-read authorization contract without depending on PostgreSQL, S3, a key-management system, or an HTTP framework.

15 Relationship to FEN

The source domain and FEN should remain siblings with a narrow reference boundary.

Concern	Source artifact domain	FEN domain
Primary purpose	Preserve evidence	Represent semantic assertions
Canonical object	Received artifact	Fact, episode, or narrative
Change model	Immutable core plus append-only observations	Supersession and correction
Location	Page, member, row, field, or byte range	Typed healthcare meaning
Provenance	Receipt and processing lineage	Citation to source and derivation
Storage	Encrypted objects and manifests	Encrypted semantic payloads

The source crate should not depend on a FEN payload crate. A FEN fact may carry or reference `SourceCitation`; the source domain does not need to know which fact types exist.

16 Repository Boundaries

The domain API should express intent while leaving infrastructure choices to adapters.

```
pub trait SourceArtifactRepository {
    type Error;

    fn append(
        &mut self,
        artifact: SourceArtifact,
    ) -> Result<(), Self::Error>;

    fn get(
        &self,
        id: &SourceArtifactId,
    ) -> Result<Option<SourceArtifact>, Self::Error>;
}

pub trait SourceObjectStore {
    type Error;

    fn put_verified(
        &mut self,
        bytes: &[u8],
        protection: &ProtectionRef,
    ) -> Result<StoredObjectRef, Self::Error>;
}
```

Production implementations will likely be asynchronous and stream large objects rather than accepting a byte slice. The simplified signatures show the domain boundary without prematurely choosing a runtime or storage SDK.

17 Decisions to Make Later

The initial model should leave these choices open:

- object-storage provider and database technology;
- exact account key hierarchy and key-management service;
- final acquisition-authority taxonomy;
- retention periods and jurisdiction-specific deletion rules;
- which extracted members are cached as independent stored objects;
- complete locator support for spreadsheets, DICOM, XML, and proprietary formats;
- whether source citations are embedded in FEN provenance or stored as typed edges; and
- parser orchestration, queueing, retries, and sandboxing technology.

Deferring these decisions keeps the first domain model stable without preventing stronger future implementations.

18 Recommended Implementation Sequence

1. Implement typed identifiers, digests, `StoredObjectRef`, and the immutable `SourceArtifact` aggregate.
2. Add constructor validation and an append-only in-memory repository.
3. Add a streaming encrypted object-store interface and a local test adapter.
4. Add bounded ZIP inspection and compound manifests using Epic EHI fixtures.
5. Add processing-run and source-citation records.
6. Add persistent adapters and mandatory access auditing.
7. Integrate citations into FEN provenance and test reprocessing with two parser versions.

This order proves preservation and lineage before investing in broad FEN conversion or transport-specific infrastructure.

A Consolidated Initial Rust Schema

The following listing collects the principal types introduced throughout the document. It is a design sketch: constructors, validation errors, serialization boundaries, and asynchronous storage APIs should be specified during implementation.

A.1 Identifiers, artifact, and stored object

```
pub struct SourceArtifactId(pub String);
pub struct StoredObjectId(pub String);
pub struct ArtifactMemberId(pub String);
pub struct ProcessingRunId(pub String);
pub struct AccountId(pub String);
pub struct CommunicationsEndpointId(pub String);
pub struct ProtectionRef(pub String);
pub struct TransportEvidenceRef(pub String);
pub struct AuthorityEvidenceRef(pub String);
pub struct Timestamp(pub String);

pub struct SourceArtifact {
    pub id: SourceArtifactId,
    pub account_id: AccountId,
    pub received_at: Timestamp,
    pub receipt: ReceiptProvenance,
    pub authorization: AcquisitionAuthority,
    pub structure: ArtifactStructure, // nests the original ref
    pub supplied_metadata: SuppliedMetadata,
}

pub enum ArtifactStructure {
    Single {
        original: StoredObjectRef,
    },
    Compound {
        original: StoredObjectRef,
```

```
        manifest: ArtifactManifest,  
    },  
}
```

A.2 Stored object integrity

```
pub struct StoredObjectRef {  
    pub object_id: StoredObjectId,  
    pub digest: ContentDigest,  
    pub byte_length: u64,  
    pub media_type: MediaType,  
    pub protection: ProtectionRef,  
}  
  
pub struct ContentDigest {  
    pub algorithm: DigestAlgorithm,  
    pub value: Vec<u8>,  
}  
  
pub enum DigestAlgorithm {  
    Sha256,  
}  
  
pub struct MediaType {  
    pub value: String,  
}
```

A.3 Compound manifest

```
pub struct ArtifactManifest {  
    pub format: ContainerFormat,  
    pub members: Vec<ArtifactMember>,  
}  
  
pub enum ContainerFormat {  
    Zip,  
    Tar,  
    GzipTar,  
    Other(String),  
}  
  
pub struct ArtifactMember {  
    pub id: ArtifactMemberId,  
    pub relative_path: String,  
    pub uncompressed_length: u64,  
    pub digest: ContentDigest,  
    pub detected_media_type: Option<MediaType>,  
    pub extracted_object: Option<StoredObjectRef>,  
}
```

A.4 Receipt provenance

```
pub struct ReceiptProvenance {
  pub channel: ReceiptChannel,
  pub endpoint_id: CommunicationsEndpointId,
  pub external_message_id: Option<String>,
  pub claimed_sender: Option<String>,
  pub transport_evidence: Vec<TransportEvidenceRef>,
}

pub enum ReceiptChannel {
  SecureUpload,
  Smtip,
  DirectMessaging,
  Fhir,
  ProviderApi,
  LocalImport,
}

pub enum AcquisitionAuthority {
  PatientProvided,
  PatientDirectedDisclosure,
  HipaaRightOfAccess,
  AuthorizedAccountManager,
  Other(AuthorityEvidenceRef),
}

pub struct SuppliedMetadata {
  pub filename: Option<String>,
  pub declared_media_type: Option<MediaType>,
  pub source_created_at: Option<Timestamp>,
}
```


A.5 Processing and citations

```
pub struct ProcessingRun {
    pub id: ProcessingRunId,
    pub artifact_id: SourceArtifactId,
    pub processor: ProcessorIdentity,
    pub configuration_ref: Option<String>,
    pub started_at: Timestamp,
    pub completed_at: Option<Timestamp>,
    pub outcome: ProcessingOutcome,
}

pub struct ProcessorIdentity {
    pub name: String,
    pub version: String,
}

pub enum ProcessingOutcome {
    Running,
    Succeeded,
    SucceededWithWarnings,
    Failed { error_code: String },
}

pub struct SourceCitation {
    pub artifact_id: SourceArtifactId,
    pub member_id: Option<ArtifactMemberId>,
    pub location: SourceLocation,
    pub processing_run_id: ProcessingRunId,
}

pub enum SourceLocation {
    WholeArtifact,
    PdfPage { page_number: u32 },
    DelimitedField {
        data_row: u64,
        column_name: String,
    },
    JsonPointer { pointer: String },
    ByteRange { start: u64, length: u64 },
}
```

B Review Checklist

Before treating this sketch as an implementation contract, confirm:

- whether a source artifact represents one receipt or a logical source assembled across receipts;
- when compound inspection is considered complete enough to finalize the artifact;
- which receipt and authority fields are mandatory for each channel;
- whether member paths and supplied filenames require field-level encryption;
- how citation locators remain stable across CSV dialects and malformed rows;
- which events are retained after a legally required purge; and
- how the future account and authority domains provide the referenced typed IDs.